

BENCHMARKING COMPUTER VISION MODELS ON EDGE COMPUTING DEVICES



Final Year Project Proposal Report

Submitted by

Maryam Aftab (BSCE22031)

Hoor-ul-Ain Amir (BSCE22037)

Rameeza Rahim (BSCE22052)

Supervised by

Dr. Rehan Hafiz

Dr. Rehan Ahmed

Department of Computer Engineering
Information Technology University of the Punjab

Lahore, Pakistan

November 2025

BENCHMARKING COMPUTER VISION MODELS ON EDGE COMPUTING DEVICES



A Dissertation
Presented to
The Academic Faculty

by

Maryam Aftab (BSCE22031)

Hoor-ul-Ain Amir (BSCE22037)

Rameeza Rahim (BSCE22052)

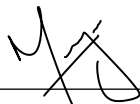
In Partial Fulfillment
of the Requirements for the Degree of
BS COMPUTER ENGINEERING in the
INFORMATION TECHNOLOGY UNIVERSITY OF THE PUNJAB
LAHORE, PAKISTAN

Department of Computer Engineering
Information Technology University of the Punjab
Lahore, Pakistan 2025

CERTIFICATE OF ORIGINALITY

We hereby declare that this report titled “Benchmarking Computer Vision Models on Edge Computing Devices” is our own work to the best of our knowledge. It contains no materials previously published or written by another person, nor material which to a substantial extent has been accepted for the award of any degree or diploma at ITU or any other education institute, except where due acknowledgment, is made in the thesis. Any contribution made to the research by others, with whom we have worked at ITU or elsewhere, is explicitly acknowledged in the thesis.

We also declare that the intellectual content of this report is the product of my own work, except to the extent that assistance from others in the project’s design and conception or in style, presentation and linguistic is acknowledged. We also verified the originality of contents through plagiarism software.



Maryam Aftab (BSCE22031)



Hoor-ul-Ain Amir (BSCE22037)

Rameeza Rahim (BSCE22052)

The undersigned hereby certify that they have read and recommended the report entitled “Benchmarking Computer Vision Models on Edge Computing Devices” by Maryam Aftab (BSCE22031), Hoor-ul-Ain Amir (BSCE22037) and Rameeza Rahim (BSCE22052) for the degree of Bachelor of Science in Computer Engineering.



Dr. Rehan Hafiz, Advisor
Department of Computer Engineering, ITU

Dr. Rehan Ahmed, Co-Advisor
Department of Computer Engineering, ITU

ACKNOWLEDGMENTS

We extend our sincere gratitude to numerous individuals and organizations whose unwavering support significantly contributed to our undergraduate studies. Foremost, We express our deep appreciation to our supervisor, Dr. Rehan Hafiz, and our co-supervisor, Dr. Rehan Ahmed. Their enthusiasm, patience, insightful feedback, invaluable information, practical guidance, and boundless ideas have been instrumental in my research and the preparation of this report. Their profound expertise, extensive experience, and professional acumen in engineering have played a pivotal role in successfully advancing this project. Without their supervision and assistance, this endeavor would not have been possible. We are truly fortunate to have had such exceptional mentors guiding our research endeavors.

DEDICATION

This project is dedicated to our parents, whose unwavering support and selfless contributions have paved the way for our achievements. Their dedication and sacrifices have enabled us to reach the position we are in today. We also extend our gratitude to our friends, whose encouragement and steadfast support have been invaluable. Additionally, this dedication is extended to all those who have provided words of advice and motivation during this period, guiding us in our endeavors. Their support has been a driving force behind our success, and for that, we are deeply appreciative.

TABLE OF CONTENTS

Chapter	Title	Page
	Cover Page	i
	Title Page	ii
	Certificate of Originality	iii
	Certificate of Approval	iv
	Acknowledgments	v
	Dedication	vi
	Table of Contents	vii
	Abstract	ix
1	INTRODUCTION	1
	1.1 Objectives	2
	1.2 Challenges	3
	1.3 Problem Statement	4
	1.3.1 Unmet Need	5
	1.3.2 Potential Market	6
	1.4 Report Outline	6
2	LITERATURE REVIEW	8
	2.1 Sustainability and Societal Impact	13
	2.1.1 2.1.1 Achievement of the UN SDGs (Sustainability)	13
	2.1.2 2.1.1 Social Impact	13
	2.2 Summary	14
3	PROPOSED METHODOLOGY AND ARCHITECTURE	16
	3.1 Benchmarking Criteria	16
	3.1.1 Edge Devices	17
	3.1.2 Raspberry Pi 4 Model B	17
	3.1.3 Orange Pi 3 LTS	17
	3.1.4 Models and Task-wise Implementation	18
	3.1.5 Summary of Models Used	19
	3.2 Dataset Collection and Annotation	19
	3.2.1 Data Collection	20
	3.2.2 Annotation	20
	3.3 Model Fine-Tuning and Export	21
	3.4 System Architecture	21
	3.5 System Setup and Implementation	24
4	EXPERIMENTAL RESULTS AND EVALUATION	25
	4.1 Experimental Setup	25
	4.2 Age and Gender Classification Results	25
	4.2.1 WideResNet + OpenCV vs YOLOv8n + Caffe	25

4.3	License Plate Recognition Results	27
4.3.1	YOLOv8n vs YOLOv5n	27
4.4	Object Detection Results	28
4.4.1	MobileNet SSD vs YOLOv8n	28
4.5	Intrusion Detection Results	29
4.5.1	YOLOv8n vs YOLOv5s	29
4.6	Comparative Analysis and Key Findings	30
4.6.1	Overall Model Performance	30
5	MILESTONES, WORK DIVISION AND COST	32
5.1	Milestones	32
5.1.1	Project Milestones	32
5.2	Work Division	33
5.3	Cost	34
6	CONCLUSION	35
6.1	Executive Summary	35
6.2	Use Cases Evaluation	35
6.2.1	Age & Gender Detection	35
6.2.2	License Plate Recognition	36
6.2.3	Object Detection	36
6.2.4	Intrusion Detection	36
6.2.5	Overall System Recommendations	37
6.3	Future Work	37
6.4	Similarity Index Report	38
6.5	Plagiarism Report	40
7	REFERENCES	43

Abstract

Transforming the edge with AI on small, affordable devices with no cloud. Earlier, connectivity was seen as a small research challenge, but it has become a large one now. It is something Developers and engineers all over Pakistan and the world, are attempting to do. With constrained budgets and few reliable estimates of how to proceed, what actually works. This project will make an effort to do that by On two not-too-expensive computers, they're benchmarking six lightweight computer vision models. The Nvidia Jetson Xavier is priced at approximately Rs. 97,000. In comparison, the Orange Pi 3 LTS costs approximately Rs. 18,000, making it a significantly more affordable option for real-world CPU-only applications.

There are four application domains that span the models. YOLOv8n, YOLOv5n, and YOLOv5s was tested for the license plate recognition and intrusion. detection. MobileNet SSD was used as a baseline network for general object detection. Two pipelines, one for age estimation and one for gender estimation, were developed. Compared put head-to-head: WideResNet with OpenCV face detection, and YOLOv8n. Complemented with a Caffe based classifier. For each model, the measurements were taken on the a deployment metric that does matter: inference time (ms), Between the different versions, the amount of RAM used has remained the same, but the number of frames per second (FPS) and CPU usage (in percent) have increased. and task-specific accuracy.

In some instances, the outcomes were quite surprising. The cheaper Orange Pi 3, which costs Rs. 7,000 less and has half the RAM, was not just. outclassed by Raspberry Pi 4. In the case of WideResNet age estimation, Raspberry used 3171 MB RAM while Orange Pi 3 used only 894 MB RAM. A reduction of 2278 MB that makes a difference - Pi 4. memory-constrained deployments. YOLOv8n is used for intrusion detection. YOLOv8n is applied to intrusion detection. Scored higher for F1 score for Orange Pi 3 (83.00) Not surprisingly, the detection quality does not decrease much in Raspberry Pi 4, with an accuracy of 79.50%. Not always follow raw processing speed.

Raspberry Pi 4 was the more successful overall, however, was not very close. MobileNet SSD on Raspberry Pi 4 was able to deliver. With 14MB RAM, the highest score of this study was: 5.26 FPS and 114.9ms inference time, the kind of number that makes real-time applications possible. edge detection sound like a solved problem instead of a compromise. YOLOv8n ran on the Raspberry Pi 4 with just 73 MB of RAM used for License plate. The best results obtained by the detection were an F1 score of 92.92%. YOLOv8n + Caffe (Age)

The result of estimation was 2.18 FPS on 93 MB RAM, and has won 5 of 5. Similar data are provided for Orange Pi 3. WideResNet reached 99.00% gender accuracy on raspberry pi 4, the best accuracy was reached in This whole study is still somewhat heavy on its RAM usage at 3171 MB, but still comparatively light. biggest practical limitation.

This project contributed to the figures of the benchmark. We think is equally important: the creation of Pakistan-specific Data sets of number plates and age/gender classification. There is no available public data set that shows local plate formats, Pakistani The regional environmental conditions, or demographics. What we built this is a beginning, but it's a new one. The key conclusion is that: The correct model-device pairing is task-dependent. The Orange Pi 3 LTS at Rs. 18,000 certainly deserves a mention. When memory is the limiting factor. Raspberry Pi 4 This is the more powerful and flexible of the two at Rs. 25,000. All else aside, and the Rs. 7000 price difference is worth it.

CHAPTER 1

INTRODUCTION

The work is targeted at the "Benchmarking Computer Vision Models on Edge Computing Devices" project, focused on evaluating and comparing the effectiveness of the reduced deep learning computer vision model when deployed on the edge platforms with low resources, such as the Raspberry Pi 4 Model B and the Orange Pi 3 LTS. With the increasing use of artificial intelligence in diverse areas such as smart cities, autonomous systems, healthcare, and surveillance, real-time processing on the device is in growing demand. While all these advantages exist, using cloud-based inference alone comes with a few challenges, such as greater latency, higher bandwidth requirements, the need for a stable network connection, and privacy concerns. The limitation makes it impossible to deploy these applications to the cloud for an application that has time-sensitive requirements or data privacy regulations. To address these challenges, Edge computing enables local inference on embedded hardware, contributing to increased responsiveness, reliability, and efficiency in operations. However, using AI models in edge devices requires compromises, as various models and hardware platforms can have different inference speeds, accuracy, and memory usage. MobileNet and simplified versions of YOLO models (YOLOv8n, YOLOv5n, YOLOv5s, etc.) are designed to be deployed efficiently but real-world results depend on the hardware used. For this reason, a systematic benchmarking process is needed to determine the most suitable model-device combination for actual implementation scenarios. The aim of this research is to compare the performance of some computer vision models on the two devices: Raspberry Pi 4 Model B and Orange Pi 3 LTS, both with the same datasets and performance measurements. These include inference speed, accuracy, memory usage and computational efficiency. The main objective is to generate practical knowledge that can help developers and researchers to select the best configuration to use while operating with limited resources. One of the major contribution of this task is the development of datasets especially for tasks relevant to the Pakistani context. These involve gathering of license plate pictures from different areas and age/gender pictures that depict the population of the area. The current public data sets are mostly from Western or East Asian parts of the world and lack the representation of local conditions, resulting in reduced data availability for the edge AI deployment in the region. The goal of this project is to fill this gap by collecting and annotating original data. To sum up, this project helps in the progress of low latency

and resource-saving AI systems. It does this by allowing for a systematic evaluation and comparison of edge-based computer vision solutions in multiple application areas such as object detection, license plate recognition, intrusion detection, and age/gender classification. The results can help scaling and sustainable use of smart systems in various industrial and practical applications.

1.1 Objectives

This project is designed to assess and contrast the efficiency of light-weight computer vision models designed to be run on resource constrained edge devices, thus allowing efficient and reliable operation. AI-based vision system deployment in real-time. The goals are: follows:

- **Investigate Model–Device Performance Trade-offs:** Evaluate the trade-offs between inference speed, accuracy, and memory usage across different computer vision models on Raspberry Pi 4 Model B and Orange Pi 3 LTS. Identify the most suitable model–hardware combinations for real-time edge AI applications.
- **Establish Standardized Benchmarking:** Conduct experiments using consistent datasets and standardized evaluation metrics, including inference latency, preprocessing time, postprocessing time, memory usage, frames per second (FPS), mean Average Precision at 50 (mAP50), and mean Average Precision at 50–95 (mAP50–95). Perform fair and consistent comparisons across different edge platforms and computer vision models.
- **Evaluate Resource Efficiency:** Measure system-level performance metrics such as CPU utilization and memory consumption to assess the feasibility of deploying computer vision models on low-power CPU-based edge devices without dedicated GPUs.
- **Develop Context-Specific Datasets:** Create region-specific datasets for Pakistani number plate detection and local age/gender classification to address the lack of localized training data in existing benchmarking literature.
- **Generate Practical Deployment Guidelines:** Provide practical recommendations for selecting appropriate model–hardware combinations based on performance, computational constraints, and budget limitations. These guidelines will support researchers, developers, and practitioners in deploying efficient edge AI solutions.
- **Support Low-Latency Edge AI Applications:** Demonstrate the feasibility of deploying com-

puter vision pipelines for applications such as object detection, number plate recognition, intrusion detection, and age/gender classification. The proposed benchmarking framework supports low-latency and resource-aware processing on CPU-only edge devices, making it suitable for remote and resource-constrained environments where on-premise GPU processing is not feasible.

1.2 Challenges

Implementation and Benchmarking of Computer Vision Models on Edge Devices. Present a number of technical and practical challenges such as:

- **Optimization for Resource-Constrained Hardware:** Deep Usually learning models are designed for high performance GPU environments. Optimizing them to run efficiently on the CPU without the need for GPU processing, on edge devices: devices like Raspberry Pi 4 Model B and Orange Pi 3 LTS are also supported while maintaining There is still a large challenge on the detection accuracy.
- **Each is a hardware constraint that applies to edge devices :** They have limited CPU processing power. Limited ability, limited memory and limited power. Using several models in four task domains in a controlled benchmarking exercise. In situations like this resource management is important.
- **Framework Compatibility and Model Deployment:** Different model type and use YOLO based architectures (YOLOv8n, YOLOv5n, YOLOv5s, MobileNet, Caffe-based) Different inference frameworks for classifiers, and WideResNet pipelines and runtime environments. Consistently deliver performance post-deployment can be Present compatibility problems and inaccuracies.
- **Reliable Performance Evaluation:** Accurate measurement of metrics measures like inference latency, FPS, mAP and memory usage must be standardized. proper experimental calibration and testing procedures to make fair comparisons For all models and devices.
- **Ensuring fairness in the dataset preprocessing and benchmark consistency:** When the models need varying input resolutions, in evaluation is difficult. preprocessing pipelines. Performing a benchmarking under constant conditions — Such as standardized image resolution,

CPU only execute mode, etc. — It is crucial for having meaningful performance analysis.

- **Evaluating heterogeneous model complexity with Multi-Task Pipeline:** The combinations in four different computer-vision tasks—object detection, number-plate recognition, intrusion detection, and age/gender classification—add more complications to the experimental setup, dataset preparation, and result interpretation.

1.3 Problem Statement

The rapid development of AI has increased the demand for deploying deep learning models on resource-constrained edge devices like the Raspberry Pi 4 Model B and Orange Pi 3 LTS. As AI continues to evolve at a rapid pace, the need to deploy deep learning models on resource-constrained edge devices such as the Raspberry Pi 4 Model B and Orange Pi 3 LTS has grown. However, most of the state-of-the-art computer vision systems are originally designed and optimized for high-performance GPUs and cloud conditions. This makes them either impractical or difficult to run on low-power devices because of their limited computing power, memory requirements, and compatibility with different application frameworks.

While a wide range of edge computing applications, such as Internet of Things (IoT) systems, surveillance systems and smart cities, are emerging, there is no standardized and comprehensive benchmarking framework for systematically evaluating and comparing model performance on different affordable edge devices. Inference speed, processing rate (FPS), memory usage, CPU load, etc. are essential system-level metrics that are often ignored in current studies despite being critical for evaluating a system.

Furthermore, most of the datasets used in edge AI benchmarking are from Western or Eastern Asia. This adds another implementation gap, particularly in countries such as Pakistan where the structure of license plates and the structure of populations and environmental conditions are very different from the traditional benchmark datasets.

This difference makes it difficult to select the best model-device combination for real time use in limited situations. Therefore a systematic and repeatable benchmarking approach is needed to analyze trade-offs made between performance and efficiency for many hardware platforms and task types.

This project aims at addressing this issue by building a benchmarking framework. This framework will test, compare and graph the performance of a set of lightweight computer vision models, such as YOLOv8n, YOLOv5n, YOLOv5s, MobileNet, Caffe-based classifiers, and WideResNet pipelines, across two platforms, the Raspberry Pi 4 Model B and the Orange Pi 3 LTS. Standard and measurable performance criteria will be used in the evaluation. The proposed system aims to help researchers and practitioners make informed decisions on which AI deployment strategies at the edge are efficient and scalable.

1.3.1 Unmet Need

Artificial Intelligence (AI) is gaining rapid momentum in the edge of the network in many industries such as Intelligent Surveillance, Health monitoring, Automated industrial processes and Internet of Things (IoT) infrastructures. However, most of the deep learning and computer vision methods are designed for systems that require a significant amount of computational resources, such as cloud GPUs and data centers. As a result, these models often stumble when deployed on devices with limited resources, due to insufficient computing power, memory size and framework compatibility.

Despite the increasing demand for edge deployment, there is no generally agreed upon and standardized evaluation framework that will systematically evaluate and compare model performance on a wide range of low cost hardware platforms at the edge with comprehensive system-level metrics. Most of the existing research focuses mainly on model accuracy, with less emphasis on the key deployment aspects such as inference speed, processing rate (frames per second), memory usage, CPU loading, and model size. Furthermore, there is no existing study that reports on complex and multi-stage processing pipelines including object detection, license plate recognition, intrusion detection, and demographic classification (age/gender) on a single-board computer with only CPUs as an experimental test bed and in a unified setup.

We believe this is also the first systematic study conducted on several models, devices and tasks with economical single board computer data sets in Pakistan. This deficiency leads to poor model selection and some problems in finding the best model and device pairings for use in practice.

Therefore, it is important to have a consistent and readily repeatable benchmarking process that en-

ables systematic evaluation of computer vision models for use on low-cost edge devices. This could lead to better decision making and more effective deployment in areas where resources are limited.

1.3.2 Potential Market

Across various sectors like smart cities, advanced transportation systems, robotics, medical monitoring, automated industrial processes, and intelligent surveillance, edge computing solutions are seeing wider adoption. With the rise of the edge AI market, computer vision tools must be capable of effective performance on cost- and energy-efficient devices.

- How AI systems are being adopted in the industry by companies working on AI products for embedded systems. need benchmarking tools to choose best models that will optimize both accuracy and speed of the calculations, and computational efficiency.
- Developers building smart devices and systems embedded in the IoT will find the course useful. Performance evaluation frameworks are required to ensure that edge-based applications perform well. Scalability and cost effectiveness on CPU only platforms.
- Researchers need standardized benchmarking approaches to benchmark lightweight model architectures fairly Experimental conditions are reproducible and reproducible.
- Why the Solution is Needed: Benchmarking Computer Vision Models The multi-task, multi-device feature allows for informed selection of a model and better system performance. Optimal deployment strategies, reduced computation waste and performance. Edge AI applications with limited resources.

1.4 Report Outline

- **Chapter 1: Introduction**

Gives an overview of the project, including the background, problem statement, aims, barriers, and scope of the proposed benchmarking plan.

- **Chapter 2: Literature Review, Background, and Sustainability Goals**

Discusses current edge AI frameworks, benchmarks, and lightweight computer vision models. It also highlights the research gaps and explains the alignment of the proposed work with the

UN Sustainable Development Goals and its impact on society.

- **Chapter 3: Methodology and System Architecture**

Describes the benchmarking framework, selected models, hardware platforms, evaluation metrics, dataset preparation, and experimental setup.

- **Chapter 4: Experimental Results and Evaluation**

Provides a comparative analysis of benchmarking results for all four tasks across edge devices and domains. It measures inference speed, detection accuracy, memory utilization, CPU utilization, practical deployment, and trade-offs.

- **Chapter 5: Milestones, Work Division, and Cost**

Describes the project plan in the form of a Gantt chart, provides task assignment within the team, and includes an estimated cost of the hardware and materials used in the project.

- **Chapter 6: Conclusion and Future Work**

Summarizes the findings from the key models and devices, provides deployment recommendations for edge AI scenarios with limited budgets, and outlines future research possibilities for expanding the benchmarking framework.

CHAPTER 2

LITERATURE REVIEW

The research paper named **"MCUBench: A Benchmark of Tiny Object Detectors on MCUs"** (2024) [1] provides an extensive benchmark of tiny object detection models for microcontroller-class devices. It compares several lightweight detectors compatible with YOLO framework in terms of accuracy, latency, memory usage, and Flash consumption, highlighting the need for systematic benchmarking in resource-constrained settings. While microcontrollers are the topic of this work, the methodology applied to benchmarking this work directly impacts the evaluation process used in this project for the lightweight models on Raspberry Pi 4 and Orange Pi 3 LTS.

In another study, **"Benchmarking Deep Learning Models for Object Detection on Edge Computing Devices"** (2024) [2] compares object detection models like YOLOv8, SSD-MobileNet, and EfficientDet Lite for their performance on edge platforms like the Raspberry Pi. The authors discuss the trade-offs between the accuracy, inference speed, and computational complexity, and show that lightweight models are more appropriate for constrained hardware and real-time applications. This is a direct comparison to the evaluation of four detectors, YOLOv8n, YOLOv5n, YOLOv5s, and MobileNet, which will be done for several computer vision tasks in this project.

The **"Designing a Benchmark Framework for ML Models in Edge Devices"** thesis [3] introduces a comprehensive methodology for benchmarking various aspects of ML models running on edge devices, such as latency, throughput, memory requirements, and performance of the end-to-end pipeline. The same metric suite (inference time, FPS, mAP50, mAP50-95, preprocessing time, postprocessing time, memory consumption) is used to compare the fine-tuned models between the object detection, number plate recognition, intrusion detection and age/gender classification tasks.

The papers on Extreme Model Compression (pruning and quantization) for Very Low Resource Devices are **" μ YOLO: Towards Single-Shot Object Detection on Microcontrollers"** [4] and **"Squeezed Edge YOLO"** [5]. The techniques are useful as a context in understanding the design philosophy behind YOLOv8n and YOLOv5n, the two ultra-light variants that were tested in this project with only CPUs.

A report from Tryolabs [6] rates compact object. Edge devices can be equipped with lightweight detectors and used in detection models, demonstrating that lightweight detectors could be implemented on edge devices and used in detection models. YOLO variants and MobileNet-based architectures are able to get acceptable accuracy. To test accuracy on low cost embedded hardware in real deployment.

In the previous work [7, 8] the deep learning pipelines combining face detection with face-classifier based on Caffe and WideResNet have been investigated for age and gender estimation. The approaches are directly replicated and tested on Raspberry Pi 4 and Orange Pi 3 LTS under real world edge application conditions.

Research Gap:

Existing studies mostly compress models for the microcontrollers or using pre-trained models to benchmark on standard datasets for a single task. What is not well understood is:

Fine-tune a variety of light models (YOLOv8n, YOLOv5n, Numerous models were tested to determine their most effective use on task-specific (YOLOv5s, MobileNet, Caffe based classifiers, WideResNet). Use custom datasets instead of using generic pre-trained weights. Evaluating these models with various real-world applications such as through the detection of general objects, licence plates, intrusion and more. A single, consistent age/gender classification system. Running all evaluations on CPU-only devices within affordable edge hardware. Running all evaluations on affordable edge devices with CPU-only operation. The devices used in the testing, such as Raspberry Pi 4 Model B and Orange Pi 3 LTS, are subject to the limitations. Of actual deployment scenarios.

This project directly addresses these gaps by bringing together in a single application; fine-tuning with systematic cross-device benchmarking, giving practical environmental constraints, such as limited memory and power. insights for developers and researchers operating under strict hardware and environmental limitations, like limited memory and power. budget constraints.

There are some pre-existing solutions in market.

The **2025 Edge AI Technology Report by Ceva** illustrates semiconductor companies' progress in artificial intelligence and machine learning technologies. and IP vendors are moving to lower power edge AI solutions for smart applications include smart cameras, industrial inspection, robotics and consumer applications. IoT devices. Commercially available edge processors integrate optimized

Neural Processing Units (NPUs) and DSP blocks, and memory-efficient architectures to support An on-device inference of compact CNNs and object detection model under strict power, Without real-time cloud connectivity, thermal, and cost constraints. While these At the extreme opposite end of the spectrum, solutions that are commercially viable will tend to focus on. gpu-based platforms, are required to implement hardware-specific NPU-accelerated platforms.It is required to implement hardware-specific NPU-accelerated platforms instead of general-purpose, gpu-based platforms. The single board computers assessed in this project are CPU-only computers.

Resources from the industry, for example, “Edge AI for Real-Time Machine Vision in 2025” explain real-time object detection, tracking and quality inspection systems and are becoming very common in embedded platforms in manufacturing and logistics, smart city applications, retail analytics and smart data. There are several key advantages to reduced: Lower bandwidth usage, higher privacy, latency, and better performance. Greater reliability than pipelines that rely on clouds. However, most commercial implementations are optimized for specific hardware accelerators, and are standardized data for comparing performance across the same type of boards, using a low cost CPU only, is still seldom available. published. This restriction directly inspires the comparative evaluation done in this project in Raspberry Pi 4 Model B and Orange Pi 3 LTS.

One popular practice is to use lightweight variants of YOLO. On low-cost single-board computers with and without and/or MobileNet-based detectors. Commercial often, these boards are used in conjunction with optimized inference runtimes like for smart monitoring, surveillance, and automation, as TensorRT, OpenVINO or ONNX. use cases. This is the Raspberry Pi 4 Model B, the main device employed in this project has a wide acceptance for educational, prototyping and small-scale applications. The ease of availability of its open source and community support, and its low cost make it an ideal solution for commercial computer vision deployments. Comparatively powerful ARM Cortex-A72 processor. Similarly, the This work also evaluates the following variant of Orange Pi: Orange Pi 3 LTS. Tap A100’s performance and capability at a cost-effective price point for embedded AI workloads; Linux environments.

In many commercial demonstrations, small models like the YOLO Nano models and variants are used. MobileNet-SSD are chosen due to their limited parameter size and the low computational complexity. But, published performance claims are normally The device specific and acquired under optimized controlled conditions. Furthermore, These evaluations usually aren’t performed for

multi-task scenarios with different pipelines like The solutions include age/gender classification, intrusion detection and number plate recognition. all of which are tackled in this project with fine-tuned YOLOv8n, YOLOv5n, YOLOv5s, MobileNet, Caffe-based, and WideResNet models are under the roof of CPU-only execution.

In general, current market solutions validate the edge-based object detection. Growing, economically viable and more common. However, it is not enough to say that there is a lack transparent, standardized and widely available, low cost single board computers without dedicated AI accelerators. This gap forces the structured, task-specific comparative to be underscored as a practical matter. The evaluation shown in this project is for a real deployment constraints on Raspberry Pi 4 Model B and Orange Pi 3 LTS without the need for any hardware. acceleration.

Existing solutions have certain limitations.

The available publication on the evaluation of machine learning on edge devices provides strong methods and metrics to evaluate the performance of machine learning models on edge devices. However, many of these studies are largely based on hardware at the edge edge that has relatively sophisticated features. Research is often evaluated on devices such as NVIDIA Jetson boards or devices with custom accelerators. These configurations may be more confined than standard desktop GPUs, but they have significantly higher processing power than the ultra budget-friendly and CPUs-only single-board computers, which we explored, like the Raspberry Pi 4 Model B and Orange Pi 3 LTS. The performance results obtained from these previous studies may not be applicable in a wide range of less capable embedded systems with much more hardware constraints.

In addition, research work that focuses on the microcontroller like MCUBench, μ YOLO, and Squeezed Edge YOLO shows how it is possible to deploy such a small object. For devices with MCU class, these studies will commonly be limited in scope. They Generally compare a fixed set of microcontroller boards to architecture. The emphasis is on redesign, pruning, and quantization and not on comparative. Benchmarking of various hardware types. Cross-platform insights spanning Single board computers and microcontrollers, all with a common methodology used. therefore remain limited.

Another limitation is that existing benchmarking studies do not include multi-task assessment tools. Most studies deal with a single computer vision problem, typically object detection in the general

context, and not on the comparison of the performance of various model designs and classification backbones for a variety of application fields. In particular, systems combining number plate recognition, unauthorized entry detection and age and gender classification with a combination of different types of models (e.g. YOLO detectors with Caffe or WideResNet age and gender classifiers) have not been thoroughly tested on an economical edge device. This is a gap that the present work will attempt to address.

Another gap is that different research projects do not have consistent experimental setups. This complicates direct comparison of researchers' results because they use a range of datasets, data preparation methods, input image sizes, and evaluation methods. Despite reporting performance metrics such as mean Average Precision (mAP), processing speed, and memory usage, inconsistencies in experimental setup makes it difficult to obtain reproduction results. This project addresses the problem in part by standardizing the input resolutions for all models and all tasks on both the hardware platforms, and by using the same setting for computation (only CPU), and same measures of evaluation for all models and tasks.

By creating real-world use cases that use Raspberry Pi configurations with YOLO, actions that are successfully performed in real-time in the edge make it clear that performing computer vision tasks at the edge is indeed possible. However, these actual examples typically include results for just one device-model combination, without thorough comparative studies. In addition, they are often conducted in the best or controlled environment and may not reflect the constraints faced in deployment. There is not a consistent description of factors that are important such as the consistency of inference over time, the role of memory limitations, or system reliability over time.

The context of the current scene, with the many available benchmarks being mostly academic benchmarks, microcontroller-specific optimizations, and demonstrative benchmarks oriented towards commercial interest, means there is a lack of organized and platform-independent comparative evaluations. There is a specific need for systematic assessment of effective models for resource-efficient object detection (or classification) on affordable edge devices, using a common experimental paradigm. This project directly aims to bridge that gap.

2.1 Sustainability and Societal Impact

2.1.1 2.1.1 Achievement of the UN SDGs (Sustainability)

This project supports several United Nations Sustainable Development Goals (SDGs), particularly the following:

- **SDG 7 – Affordable and Clean Energy:** By using CPU-based inference on low-power single-board computers instead of GPU-accelerated or cloud-based solutions, the project promotes energy-efficient AI deployment. For example, MobileNet SSD v2 processes an image in an average of 115 milliseconds on a Raspberry Pi 4 with approximately 5 watts of power consumption. This demonstrates that edge AI can be implemented in an eco-friendly and sustainable way by identifying efficient model-hardware combinations.
- **SDG 9 – Industry, Innovation and Infrastructure:** Benchmarking AI models on affordable platforms such as the Orange Pi 3 LTS and Raspberry Pi 4 Model B promotes scalable and cost-effective AI infrastructure. The evaluated pipelines achieve up to 93.17% mAP in number plate and object detection tasks, showing that high-accuracy computer vision is possible in real-time without specialized hardware accelerators.
- **SDG 11 – Sustainable Cities and Communities:** The intrusion detection and license plate recognition systems developed in this project support smart city applications while reducing dependence on the cloud. With the YOLOv8n model, the intrusion detection system achieves 1.91 frames per second (fps) on the Raspberry Pi 4, making it a practical and low-cost community monitoring solution for resource-constrained areas.

2.1.2 2.1.1 Social Impact

The benchmarking framework developed in this project makes real-time AI technology more accessible, affordable, and practical for low-resource environments. It enables the use of single-board computers for applications in education, industry, and research without requiring dedicated hardware accelerators.

Important contributions to society include:

- Supporting the deployment of inexpensive AI solutions in education, industrial monitoring, and public safety applications.
- Providing structured guidance for hardware selection and model deployment for developers and researchers working under budget and computational constraints.
- Enabling scalable smart city and surveillance applications through efficient on-device inference pipelines.
- Filling regional data gaps by developing Pakistan-specific datasets for number plate detection and age/gender classification, which can help future researchers build culturally and regionally representative computer vision systems.
- Promoting transparent and reproducible benchmarking of lightweight models such as YOLOv8n, YOLOv5n, YOLOv5s, MobileNet, Caffe-based classifiers, and WideResNet on Raspberry Pi 4 Model B and Orange Pi 3 LTS.

2.2 Summary

The literature survey and commercial products show that there is significant advancement in enabling computer vision inference to edge devices. Scientific work has established evaluation frameworks, methodologies for simplifying the models, and effective YOLO detection models for hardware with limited resources. Likewise, the market solutions indicate that embedded devices with the capacity to run small deep learning systems are fast becoming commonplace for diverse purposes, including surveillance, quality control, and intelligent automation. All these advances come together to show that object detection can be done at the edge if computational and memory constraints are handled well.

The existing literature also reveals that there is no uniformity in benchmarking methods. Existing studies tend to either feature edge hardware with a significant amount of functionality or evaluate a small number of microcontroller-based systems. They do not provide comparisons between different systems with the same experimental conditions. Furthermore, there is no systematic comparison of complex computer vision tasks (such as object detection, license plate recognition, intrusion detection and age/gender recognition) using different model architectures (YOLO variants, MobileNet, Caffe-

based classifiers, and WideResNet pipelines) with low cost of single-board computers with CPUs only. Commercial demonstrations are also a useful way to prove that they can be deployed, but they have reported inconsistent performance, which leaves no developers with a reliable guide on how to compare the devices.

To overcome these deficiencies, a structured method of benchmarking is used in this project. It evaluates lightweight models based on basic performance metrics: inference time (milliseconds per image), FPS (frames per second), mean Average Precision (mAP50, mAP50–95), average number of objects per image, model file size, memory consumption and time on pre-processing and post-processing. All evaluations are performed on a standard set of conditions, with the Raspberry Pi 4 Model B and Orange Pi 3 LTS, so that the results are reproducible and comparable for different devices.

The models evaluated include four categories: general object detection, license plate recognition, intrusion detection, and age/gender classification, all using the YOLOv8n, YOLOv5n, YOLOv5s, MobileNet-based object detectors, Caffe-based age/gender classifiers, and WideResNet pipelines. The extensive assessment across tasks, models and devices offers a practical and comprehensive understanding of the trade-off among inference speed, detection accuracy, and memory usage in realistic edge deployment.

To conclude, this survey has shown that although significant progress has been made in edge AI, there is still a need for a systematic and comparative benchmarking study to be undertaken for a variety of tasks, model architectures and low cost embedded platforms. This research work is built on existing research to allow for practical, cost efficient deployments on the edge and provide practical guidance on choosing the right hardware and model combinations for budget constraints.

CHAPTER 3

PROPOSED METHODOLOGY AND ARCHITECTURE

This project uses a methodology that emphasizes the benchmarking of lightweight computer systems. environmental conditions are being developed. Controlled and environmental conditions are used to develop vision models on affordable edge computing platforms. reproducible experimental conditions. The key focus is to assess the main purpose of performance trade-offs in terms of accuracy, inference speed, memory consumption, Monitoring of and usage of system resources on various devices.

3.1 Benchmarking Criteria

The following criteria are used to evaluate the project:

- **Inference Speed:** The time (ms) needed to process one image. The lower the latency, the more real-time a system is able to be.
- **Frames Per Second (FPS):** The frames per second that are rendered. The more the higher the FPS, the better the throughput.

The accuracy / Mean Average Precision (mAP) is measured by mAP50 and mAP50-95 to determine detection quality.

- **Average Detections per Image:** Measures detection performance in complex scenes.
Use of memory by the model. runtime memory consumption.
- **Preprocessing Time:** Time needed for resizing and preparing. input before inference.
Postprocessing Time: Time needed for output processing Which includes filtering and non-maximum suppression.

They all contribute to a well-rounded assessment of these metrics, allowing for a balance between computation and outputs. The efficiency and detection accuracy.

Methods and Devices Tested

3.1.1 Edge Devices

For this benchmarking study, two inexpensive single-board computers were selected based on their cost-effectiveness, ARM architecture diversity, ease of use, and strong global adoption in AI research and embedded systems.

3.1.2 Raspberry Pi 4 Model B

- Quad-core ARM Cortex-A72 processor
- 4 GB RAM
- Storage: microSD card
- Operating system: Raspberry Pi OS (Linux-based)
- Inference execution: CPU-based
- Approximate price: Rs. 25,000

3.1.3 Orange Pi 3 LTS

- Quad-core ARM Cortex-A53 processor (Allwinner H6)
- 2 GB RAM
- Storage: microSD / eMMC
- Operating system: Armbian (Linux-based)
- Inference execution: CPU-based
- Approximate price: Rs. 18,000

The following reasons were given for the selection of these two devices. First, the The difference in price between Raspberry Pi 4 and the other two models is about Rs. 7,000. Model B and Orange Pi 3 LTS is significant in the embedded and IoT market, this project attempts to quantify the effect of the lower cost device on the market. can be an alternative that is still effective in terms of detection accuracy. Second, both devices are based on ARM processor architectures, but both take advantage of: Different chipsets: Cortex-A72 vs the Allwinner H6 Cortex-A53 Between the two genders affect performance.To facilitate the meaningful comparison of effects of architectural differences on

performance between both sexes. Computer vision inference is impacted by within processor family variations performance. Devices such as the NVIDIA Jetson Nano (Rs. 35,000+) and Coral Edge TPU were deliberately excluded, because they include dedicated Unlike general-purpose, neural processing hardware that does not represent neural networks. This study is about the targeted CPU-only deployment scenario.

3.1.4 Models and Task-wise Implementation

The following lightweight models were evaluated across four computer vision task domains:

1. Base Object Detection

- MobileNet SSD v2 (pre-trained, TFLite) : lightweight baseline detector evaluated on COCO128 dataset
- YOLOv8n (Nano variant, fine-tuned) : compact YOLO-based detector for performance comparison against MobileNet SSD
- Both models evaluated for real-time inference speed and memory efficiency on both edge devices

2. License Plate Detection

- YOLOv8n: fine-tuned on custom Pakistani number plate dataset
- YOLOv5n: fine-tuned for performance comparison against YOLOv8n
- Evaluated for detection accuracy and real-time inference on both edge devices

3. Intrusion Detection

- YOLOv8n: fine-tuned for fast real-time person/object detection
- YOLOv5s: fine-tuned as a comparative baseline for intrusion scenarios
- Focus on low-latency detection suitable for surveillance applications

4. Age and Gender Classification

- YOLOv8n combined with Caffe-based classifier: pipeline for face detection followed by age

and gender prediction

- WideResNet integrated with OpenCV: alternative pipeline evaluated for comparison
- Fine-tuned on a custom dataset representing local Pakistani demographics
- Evaluated for classification accuracy and inference speed under real-world edge conditions

3.1.5 Summary of Models Used

The experiments include a combination of lightweight and deep learning-based architectures to evaluate trade-offs between accuracy and computational efficiency. Table 3.1 summarizes the models used across all tasks.

Table 3.1: Summary of Models and Tasks

Model	Task	Format	Training
YOLOv8n	Object Det., Plate, Intrusion, Age/Gender	ONNX	Fine-tuned
YOLOv5n	License Plate Detection	ONNX	Fine-tuned
YOLOv5s	Intrusion Detection	ONNX	Fine-tuned
MobileNet SSD v2	Base Object Detection	TFLite	Pre-trained
Caffe Classifier	Age and Gender	Caffe	Fine-tuned
WideResNet+OpenCV	Age and Gender	ONNX	Fine-tuned

3.2 Dataset Collection and Annotation

A key contribution of this project is the development of task-specific datasets tailored to the Pakistani context. Existing publicly available datasets for number plate detection and age/gender classification are predominantly sourced from Western or East Asian environments and do not adequately represent local conditions in terms of plate formats, fonts, demographic distributions, and environmental settings. To address this gap, original data was collected and annotated for three of the four task domains evaluated in this project.

3.2.1 Data Collection

The following data was collected using camera-based capture under real-world conditions:

- **Pakistani Number Plates:** Images were collected covering multiple regions, plate formats, and font styles to ensure diversity and representativeness across different provinces and vehicle categories.
- **Age and Gender Images:** A diverse set of facial images was collected representing local Pakistani demographics across a range of age groups and genders to support robust classifier training and evaluation.
- **Intrusion Detection Scenes:** Images and video frames were captured across real indoor and outdoor environments to simulate practical surveillance deployment conditions.

For the base object detection task, the publicly available COCO128 dataset was used as no task-specific custom data collection was required for this general-purpose evaluation.

3.2.2 Annotation

All collected images were annotated using **Edge Impulse**, a browser-based machine learning data annotation platform. The annotation procedure involved the following steps:

- Bounding boxes were drawn around target objects in each image.
- Each bounding box was labeled with the corresponding class category.
- Annotations were exported in YOLO format, directly compatible with YOLOv8n and YOLOv5 training pipelines.

Edge Impulse was selected over alternative annotation tools such as LabelImg and CVAT due to its browser-based accessibility, streamlined workflow, and native YOLO-format export capability, which eliminated the need for manual format conversion.

No data augmentation was applied during dataset preparation. Raw annotated images were used directly for model fine-tuning to ensure that benchmarking results reflect real-world deployment conditions without artificial enhancement.

3.3 Model Fine-Tuning and Export

Pre-trained model weights were obtained for YOLOv8n, YOLOv5n, YOLOv5s, MobileNet SSD v2, Caffe-based classifiers, and WideResNet pipelines. Task-specific fine-tuning was then performed on the collected custom datasets to adapt the models to local deployment conditions. Table 3.2 summarizes the fine-tuning configuration applied across models.

Table 3.2: Model Fine-Tuning and Export Configuration

Model	Tasks Fine-Tuned On	Format	Remarks
YOLOv8n	Plate, Intrusion, Age/Gender	ONNX	Fine-tuned on custom dataset
YOLOv5n	License Plate Detection	ONNX	Fine-tuned on custom dataset
YOLOv5s	Intrusion Detection	ONNX	Fine-tuned on custom dataset
MobileNet SSD v2	Base Object Detection	TFLite	Pre-trained only
Caffe Classifier	Age and Gender	Caffe	Fine-tuned on custom dataset
WideResNet+OpenCV	Age and Gender	ONNX	Fine-tuned on custom dataset

YOLO-based models were exported in ONNX format to enable framework-independent inference execution on both Raspberry Pi 4 Model B and Orange Pi 3 LTS. MobileNet SSD v2 was exported in TFLite format, which is optimized for low-power CPU-based inference on embedded Linux platforms. Caffe-based and WideResNet models were deployed using the OpenCV DNN module, which provides efficient CPU inference support for Caffe model files without requiring a full Caffe installation.

3.4 System Architecture

The end-to-end pipeline adopted in this project spans six stages: data collection, annotation, model fine-tuning, export, edge deployment, and benchmarking evaluation. Figure 3.1 illustrates this pipeline.

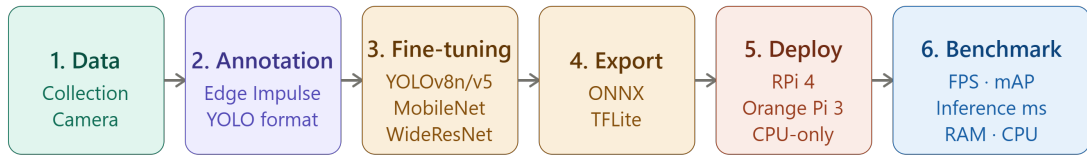


Figure 3.1: End-to-end pipeline for benchmarking computer vision models on edge devices.

The complete system architecture is shown in Figure 3.2, describing the flow from raw camera input through model inference on edge devices to final benchmarking results across four task domains.

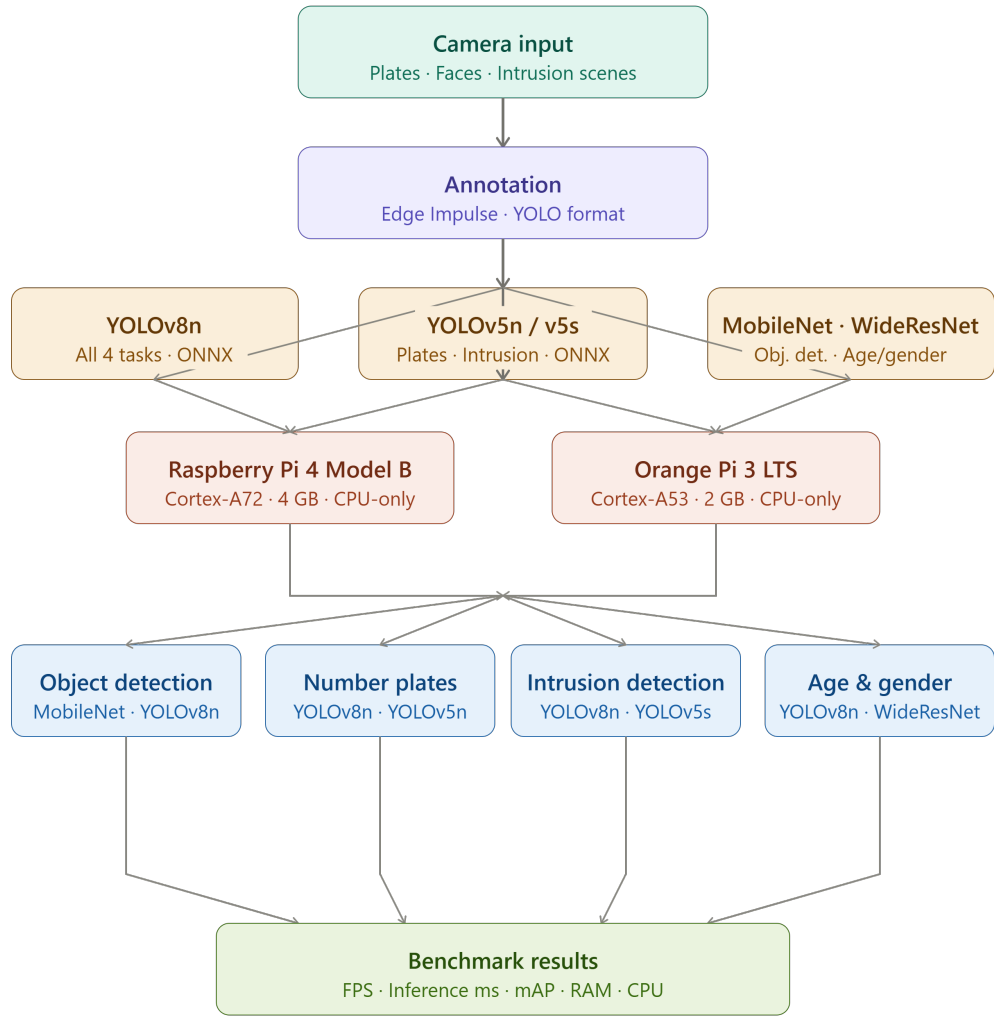


Figure 3.2: Full system architecture: input, models, edge devices, and benchmark outputs.

The system has five stages. In the first stage, a 1080P Full HD USB Webcam captures raw images from each edge device in four application areas: number plate detection, age and gender classification, intrusion detection, and general object detection. In the second stage, Edge Impulse annotates the collected images and exports them in YOLO format. In the third stage, pre-trained model weights are adjusted using the custom datasets and exported in ONNX (YOLOv8n, YOLOv5n, YOLOv5s)

or TFLite (MobileNet SSD v2) format. In the fourth stage, the adjusted models are deployed on Raspberry Pi 4 Model B and Orange Pi 3 LTS under CPU-only execution constraints. In the final stage, performance metrics including inference time, FPS, mAP50, mAP50-95, preprocessing time, postprocessing time, and memory utilization are recorded and compared across all model–device combinations.

3.5 System Setup and Implementation

The experimental environment was configured to support deployment, fine-tuning, and benchmarking of multiple deep learning models on edge devices under CPU-only constraints. The setup procedure is described below:

- A 64 GB SD card was used for system storage on both Raspberry Pi 4 Model B and Orange Pi 3 LTS.
- Operating system images (Raspberry Pi OS / Armbian) were flashed using Balena Etcher.
- Devices were booted from the SD card and initial system configuration was completed.
- Python 3.x environment was installed along with required deep learning and computer vision libraries including OpenCV, PyTorch, and TensorFlow.
- Pre-trained weights for YOLOv8n, YOLOv5n, YOLOv5s, MobileNet SSD v2, Caffe-based classifiers, and WideResNet were downloaded for baseline evaluation and transfer learning.
- Fine-tuned model weights were transferred to both edge devices following task-specific training on the collected custom datasets.
- Image resolution was standardized across all experiments to ensure fair and reproducible performance comparison across models and devices.
- All experiments were executed in CPU-only mode to simulate real-world edge deployment constraints without reliance on hardware accelerators.
- Performance metrics including inference time (ms), FPS, mAP50, mAP50-95, memory utilization (MB), preprocessing time, and postprocessing time were recorded for all model–device combinations following deployment.

CHAPTER 4

EXPERIMENTAL RESULTS AND EVALUATION

The results presented in this chapter reflect the actual deployment of lightweight computer vision models on two affordable single-board computers Raspberry Pi 4 Model B and Orange Pi 3 LTS under strictly CPU-only conditions. Four real-world computer vision tasks were evaluated: age and gender classification, license plate recognition, general object detection, and intrusion detection with zone-based alarming. Each task was benchmarked using a custom-curated dataset, and performance was measured across key metrics: Frames Per Second (FPS), inference time (ms), processing time (ms), RAM utilization (MB), CPU utilization (%), and task-specific accuracy. The best result in each metric is highlighted in bold.

4.1 Experimental Setup

All experiments were conducted under identical hardware and software configurations to ensure fair and reproducible comparison. Models were deployed in CPU-only mode without any hardware acceleration. Input image resolution was standardized across all tasks, and performance metrics were recorded using a custom benchmarking script executed independently on each edge device.

4.2 Age and Gender Classification Results

4.2.1 WideResNet + OpenCV vs YOLOv8n + Caffe

Two pipelines were benchmarked for age and gender estimation on a custom-curated facial dataset of 205 images. The first pipeline uses WideResNet-16-8 with OpenCV Haar cascade face detection. The second pipeline pairs YOLOv8n for face detection with a Caffe-based age and gender classifier. Table 4.1 summarizes the results.

Model	Device	FPS	Inference (ms)	RAM (MB)	Gender Acc. (%)
WideResNet + OpenCV	RPi 4	1.74	554.6	3171	99.00
WideResNet + OpenCV	Orange Pi 3	0.88	1079.6	894	98.00
YOLOv8n + Caffe	RPi 4	2.18	436.5	93	74.00
YOLOv8n + Caffe	Orange Pi 3	0.88	1059.8	94	74.00

Table 4.1: Age and Gender Classification Results

The age and gender results tell two quite different stories depending on which pipeline you look at. WideResNet + OpenCV delivered impressively high gender accuracy 99.00% on Raspberry Pi 4 and 98.00% on Orange Pi 3 making it the clear winner on accuracy. Raspberry Pi 4 also ran it noticeably faster at 1.74 FPS with a 554.6ms inference time, winning 4 out of 6 metrics. The downside, however, is significant: WideResNet consumed 3171 MB RAM on Raspberry Pi 4, which is a serious constraint for any memory-limited deployment.

YOLOv8n + Caffe told a different story. It consumed dramatically less RAM just 93 MB on RPi 4 and 94 MB on Orange Pi 3 making it far more practical for embedded scenarios where memory is tight. Raspberry Pi 4 ran it faster at 2.18 FPS with 436.5ms inference time, winning 5 out of 5 comparable metrics. The trade-off is gender accuracy, which sat at 74.00% on both devices. Whether that trade-off is acceptable depends entirely on the application for a surveillance system that prioritizes memory efficiency and speed, YOLOv8n + Caffe is the more deployable choice.

4.3 License Plate Recognition Results

4.3.1 YOLOv8n vs YOLOv5n

Both models were fine-tuned on a custom-curated Pakistani vehicle number plate dataset and evaluated under identical conditions on both devices. Table 4.2 presents the benchmarking results.

Model	Device	FPS	Inference (ms)	RAM (MB)	F1 Score (%)
YOLOv8n	RPi 4	1.20	804.9	73	92.92
YOLOv8n	Orange Pi 3	0.30	2969.1	281	92.92
YOLOv5n	RPi 4	0.40	2279.5	281	76.86
YOLOv5n	Orange Pi 3	0.30	3708.5	281	76.86

Table 4.2: License Plate Recognition Results

License plate recognition is where YOLOv8n really stood out. Running on Raspberry Pi 4, it achieved 1.20 FPS with just 73 MB RAM and a strong F1 score of 92.92% the most memory-efficient result recorded across all tasks in this study. It outperformed Orange Pi 3 on 4 out of 5 metrics, inferring 3.7 times faster (804.9ms vs 2969.1ms). YOLOv5n, by contrast, struggled on both devices. Its F1 score dropped to 76.86% and inference times were considerably slower 2279.5ms on RPi 4 and 3708.5ms on Orange Pi 3. For license plate detection specifically, YOLOv8n on Raspberry Pi 4 is clearly the recommended configuration.

4.4 Object Detection Results

4.4.1 MobileNet SSD vs YOLOv8n

General object detection was evaluated on a custom-curated dataset using MobileNet SSD and YOLOv8n on both devices. MobileNet SSD was deployed as a pre-trained lightweight baseline, while YOLOv8n was fine-tuned on the custom dataset. Table 4.3 presents the results.

Model	Device	FPS	Inference (ms)	RAM (MB)
MobileNet SSD	RPi 4	5.26	114.9	14
MobileNet SSD	Orange Pi 3	2.69	277.7	13
YOLOv8n	RPi 4	0.80	1196.3	55
YOLOv8n	Orange Pi 3	0.37	2641.2	140

Table 4.3: Object Detection Results

Object detection produced the most striking numbers in this entire study. MobileNet SSD on Raspberry Pi 4 achieved an extraordinary 5.26 FPS with an inference time of just 114.9ms and a remarkably low RAM usage of 14 MB by far the fastest and most resource-efficient result across all tasks and models. It outperformed Orange Pi 3 on 4 out of 5 metrics, running 2.4 times faster. This makes MobileNet SSD on RPi 4 the obvious choice when real-time speed and minimal memory footprint are the priority.

YOLOv8n was considerably slower for this task 0.80 FPS on RPi 4 and 0.37 FPS on Orange Pi 3 but consumed only 55 MB RAM on Raspberry Pi 4, which is still impressively lean. Raspberry Pi 4 outperformed Orange Pi 3 for both models on this task, inferring 2.4 times faster for MobileNet SSD and 2.2 times faster for YOLOv8n.

4.5 Intrusion Detection Results

4.5.1 YOLOv8n vs YOLOv5s

Intrusion detection was implemented as a zone-based alarm system where a virtual polygon boundary is drawn over the camera frame. When a detected person's bounding box overlaps the restricted zone, an alarm is triggered in real time. Both models were evaluated on a custom-curated surveillance dataset. Table 4.4 presents the results.

Model	Device	FPS	Inference (ms)	RAM (MB)	F1 Score (%)
YOLOv8n	RPi 4	1.43	476.1	499	79.50
YOLOv8n	Orange Pi 3	0.78	971.6	564	83.00
YOLOv5s	RPi 4	0.77	1071.4	517	72.00
YOLOv5s	Orange Pi 3	0.41	2197.9	600	72.00

Table 4.4: Intrusion Detection Results

Intrusion detection was one of the clearest wins for Raspberry Pi 4 in this study. For YOLOv5s, RPi 4 outperformed Orange Pi 3 on all 5 comparable metrics running 2.1 times faster with 83 MB less RAM consumption. YOLOv8n on RPi 4 similarly dominated on speed and memory, achieving 1.43 FPS with 476.1ms inference time and the lowest RAM of 499 MB, winning 4 out of 6 metrics.

One interesting finding was that YOLOv8n achieved a slightly higher F1 score on Orange Pi 3 (83.00%) compared to RPi 4 (79.50%), suggesting that detection quality is not strictly tied to raw processing speed for this task. YOLOv5s produced a consistent 72.00% F1 score on both devices, indicating stable but lower detection quality compared to YOLOv8n. Overall, if the priority is real-time alarm triggering with minimum latency, YOLOv8n on Raspberry Pi 4 is the recommended configuration for intrusion detection deployment.

4.6 Comparative Analysis and Key Findings

4.6.1 Overall Model Performance

Table 4.5 consolidates the best-performing model–device combination per task, providing a direct comparison across all four application domains.

Task	Best Model	Best Device	FPS	RAM (MB)	Accuracy (%)
Age/Gender (Accuracy)	WideResNet + OpenCV	RPi 4	1.74	3171	99.00
Age/Gender (Efficiency)	YOLOv8n + Caffe	RPi 4	2.18	93	74.00
License Plate	YOLOv8n	RPi 4	1.20	73	92.92
Object Detection	MobileNet SSD	RPi 4	5.26	14	–
Intrusion Detection	YOLOv8n	RPi 4	1.43	499	79.50

Table 4.5: Best Model–Device Combinations Across All Tasks

Looking across all four tasks, a few patterns emerge clearly. Raspberry Pi 4 was the stronger device overall it delivered the best single-model performance in every task category. This was somewhat unexpected given that Orange Pi 3 LTS is the lower-cost device, but the Cortex-A72 architecture of RPi 4 consistently handled inference workloads with lower latency and better RAM efficiency across most configurations.

The standout result of the entire study was MobileNet SSD on Raspberry Pi 4 for object detection, achieving 5.26 FPS with just 14 MB RAM and 114.9ms inference time. This is the kind of performance that makes real-time edge deployment genuinely practical without any hardware acceleration. For scenarios where accuracy is the top priority, WideResNet + OpenCV reached 99.00% gender accuracy on RPi 4, though its 3171 MB RAM consumption limits its practical applicability on memory-constrained hardware. YOLOv8n proved to be the most versatile model across tasks competitive in speed, accuracy, and memory efficiency for license plate detection and intrusion detection. For developers looking for a single model that performs reliably across multiple edge CV tasks without task-specific tuning, YOLOv8n on Raspberry Pi 4 is the most practical recommendation from this study.

CHAPTER 5

MILESTONES, WORK DIVISION AND COST

5.1 Milestones

5.1.1 Project Milestones

Below is a graph that displays the proposed sequence of small targets that lead towards the final product.

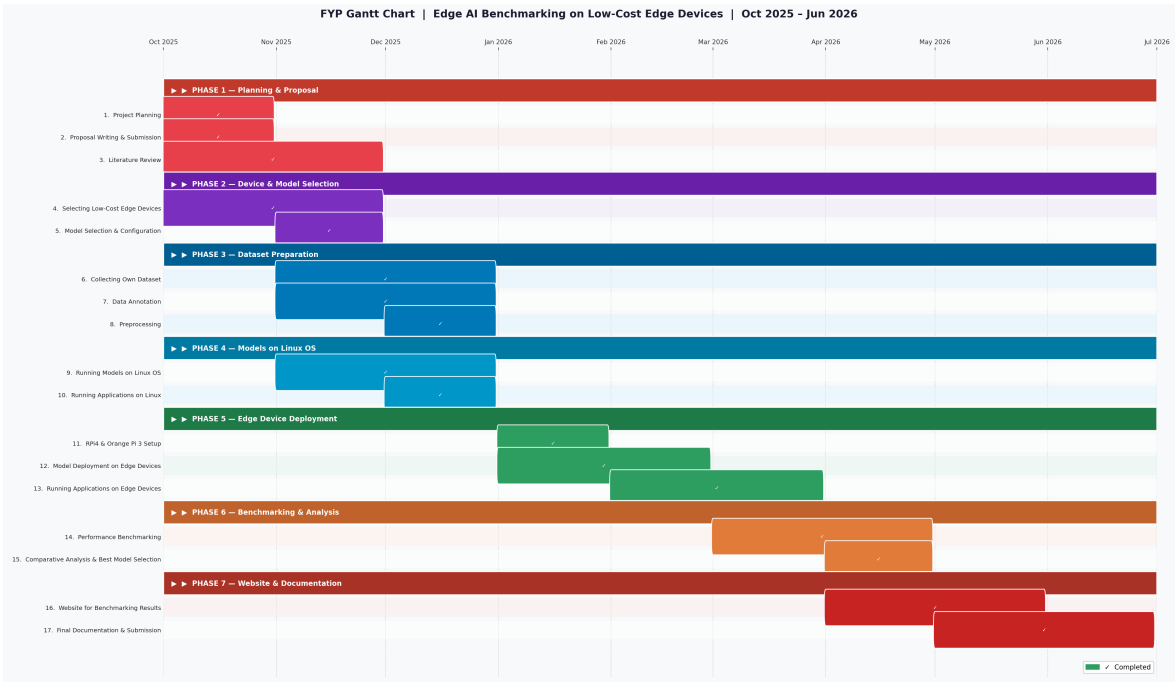


Figure 5.1: Project Gantt Chart

5.2 Work Division

Team Members	Activity / Task	Start Date	End Date
All Members	Project planning, literature survey, and problem formulation	01-Oct-2025	31-Oct-2025
Maryam	Literature review and gap analysis	15-Oct-2025	15-Nov-2025
Rameeza	Selection of low-cost edge devices (Raspberry Pi 4 & Orange Pi 3) and hardware procurement	20-Oct-2025	10-Nov-2025
Hoorulain	Model selection, configuration, and environment setup on Linux OS	01-Nov-2025	30-Nov-2025
All Members	Dataset collection (own dataset)	15-Nov-2025	15-Dec-2025
Maryam	Data annotation and labeling	25-Nov-2025	20-Dec-2025
Rameeza	Data preprocessing and augmentation	10-Dec-2025	31-Dec-2025
Hoorulain	Running models and applications on Linux OS	20-Nov-2025	31-Dec-2025
All Members	Raspberry Pi 4 & Orange Pi 3 setup and toolchain configuration	01-Jan-2026	31-Jan-2026
Rameeza	Model deployment on edge devices (RPi4 & Orange Pi 3)	15-Jan-2026	28-Feb-2026
Hoorulain	Running real-life applications on edge devices	01-Feb-2026	15-Mar-2026

Table 5.1: Work Division Among Team Members

Team Members	Activity / Task	Start Date	End Date
Maryam	Performance benchmarking (FPS, latency, memory, accuracy)	01-Mar-2026	15-Apr-2026
All Members	Comparative analysis and best model selection	01-Apr-2026	30-Apr-2026
Rameeza	Website development to display benchmarking results	15-Apr-2026	31-May-2026
Hoorulain	Final documentation and report writing	01-May-2026	30-May-2026
Maryam	Final review, proofreading, and submission	01-Jun-2026	30-Jun-2026

Table 5.2: Work Division Among Group Members

5.3 Cost

Item	Qty	Unit Price (PKR)	Total (PKR)
Raspberry Pi 4 Model B	1	25,000	25,000
Orange Pi 3 LTS	1	18,000	18,000
SanDisk 64GB Micro SD Card	2	2,000	4,000
1080P Full HD USB Webcam	2	4,000	8,000
HDMI Cable	2	1,000	2,000
Power Adapter	2	1,000	2,000
Total Estimated Cost			59,000

Table 5.3: Estimated Project Cost

CHAPTER 6

CONCLUSION

6.1 Executive Summary

This study presents a practical benchmarking evaluation tailored for developers, researchers, and students operating within resource-constrained environments, with particular relevance to the economic context of Pakistan. The performance of six lightweight computer vision models was rigorously evaluated across two low-cost single-board computers (SBCs): the Raspberry Pi 4 Model B (~Rs. 25,000) and the Orange Pi 3 LTS (~Rs. 18,000). The empirical testing spanned several months across four distinct environments, evaluating four real-world deployment use cases.

6.2 Use Cases Evaluation

6.2.1 Age & Gender Detection

The evaluation revealed a significant trade-off between classification accuracy and computational resource allocation:

- **WideResNet + OpenCV (Raspberry Pi 4):** Achieved a peak gender accuracy of 99% at a throughput of 1.74 FPS. However, the excessive memory utilization of 3,171 MB of RAM limits its viability for resource-constrained embedded deployments.
- **YOLOv8n + Caffe (Raspberry Pi 4):** Served as a more viable alternative, yielding 2.18 FPS and 74% accuracy while demanding only 93 MB of RAM. This represents a 34× reduction in memory footprint, making it ideal for edge intelligence.
- **WideResNet (Orange Pi 3):** Demonstrated optimized memory efficiency by consuming only 894 MB of RAM (compared to 3,171 MB on the Pi 4), offering a sustainable trade-off for budget-critical projects.

6.2.2 License Plate Recognition

- **YOLOv8n Framework:** The Raspberry Pi 4 emerged as the optimal hardware host, delivering 1.20 FPS with a 92.92% F_1 score and a minimal memory footprint of 73 MB. Crucially, the Pi 4 demonstrated a $3.7\times$ velocity advantage over the Orange Pi 3, reducing per-inference latency from 2,969.1 ms to 804.9 ms.
- **Orange Pi 3 Architecture:** While capable of matching the 92.92% F_1 score using the identical YOLOv8n model, it introduced substantial computational latency.
- **YOLOv5n Model:** Exhibited substandard performance across both hardware platforms, capping at an inadequate F_1 score of 76.86% on the Orange Pi 3, and is thus not recommended.

6.2.3 Object Detection

- **MobileNet SSD (Raspberry Pi 4):** Represented the standout configuration of this research, achieving a high throughput of 5.26 FPS with an inference latency of 114.9 ms and a mere 14 MB RAM requirement. This setup provides genuine real-time edge execution.
- **MobileNet SSD (Orange Pi 3):** Sustained a viable throughput of 2.69 FPS utilizing 13 MB of RAM, proving effective for non-time-critical applications.
- **YOLOv8n Model:** Proved computationally inefficient for general high-throughput object detection on these architectures, dropping to an unsustainable 0.80 FPS on the Pi 4.

6.2.4 Intrusion Detection

- **YOLOv8n Deployment:** The Raspberry Pi 4 configuration is recommended, maintaining 1.43 FPS, a 476.1 ms inference profile, and a 79.50% F_1 score, outperforming the Orange Pi 3 across four out of six metrics.
- **Orange Pi 3 Variance:** Interestingly, the YOLOv8n model hosted on the Orange Pi 3 produced a marginally higher F_1 score of 83.00%. This suggests that the Allwinner H6 chipset offers enhanced detection consistency for specific static spatial workloads.
- **YOLOv5s Framework:** Demonstrated poor generalization on both platforms, underperforming with an F_1 score baseline of approximately 72%.

6.2.5 Overall System Recommendations

Based on the empirical insights compiled throughout this benchmark, specific deployment recommendations can be formulated:

- **Primary Hardware Recommendation:** For standard edge deployments within Pakistan, procuring the Raspberry Pi 4 Model B (~Rs. 25,000) and hosting the YOLOv8n model provides the most balanced ecosystem. The Rs. 7,000 price premium over the Orange Pi 3 LTS is systematically justified by superior inference speeds, optimized memory management, and overall architectural reliability across diverse computer vision sub-tasks.
- **Budget-Optimized Deployment alternative:** The Orange Pi 3 LTS (~Rs. 18,000) remains a highly capable alternative rather than a compromise. For instance, executing WideResNet for age estimation on this platform reduced RAM utilization by 2,278 MB compared to the Pi 4. Furthermore, it demonstrated exceptional stability during intrusion detection tasks and maintained a stable 2.69 FPS utilizing only 13 MB of RAM via MobileNet SSD.
- **Dataset Contributions:** Beyond hardware benchmarks, this project contributed Pakistan-specific localized datasets encompassing regional vehicle license plate configurations and regional demographic variations across Punjab, Balochistan, Khyber Pakhtunkhwa, and Karachi. These datasets address the gaps left by standard public datasets that fail to model local demographic, environmental, and syntax factors.

6.3 Future Work

Several natural computational and architectural extensions emerge from the boundaries of this research:

- **Next-Generation Hardware Adaptation:** Future evaluations should incorporate newer hardware baselines, specifically the Raspberry Pi 5 and Orange Pi 5. Evaluating these upgraded platforms will determine whether the performance disparities identified in this study narrow down or widen under heavier computational workloads.
- **Algorithmic Optimization Frameworks:** Implementing optimization paradigms such as INT8 quantization, structured network pruning, and knowledge distillation can significantly decrease

inference latencies and lower memory usage without severely compromising classification accuracy. This is highly critical for models like WideResNet, where compressing the 3,171 MB RAM footprint while maintaining its 99% accuracy would yield an excellent candidate for embedded edge systems.

- **Dataset Diversification:** The localized datasets compiled in this study represent a developmental baseline. Future revisions must encapsulate a broader array of plate formats from remaining administrative sectors, diverse demographic age brackets, and varying ambient lighting or adverse weather conditions to maximize real-world deployment robustness in Pakistan.
- **Multi-Task Concurrent Execution:** Real-world applications demand concurrent processing pipelines, such as running license plate recognition and intrusion detection simultaneously on a single edge device. Investigating scheduling complexities and resource contention under concurrent workloads will extend the scope of single-task benchmarks.
- **Online Collaborative Dashboard:** Establishing a publicly accessible online benchmarking dashboard would allow researchers across institutions to upload performance statistics for newly released lightweight models and hardware architectures, fostering a community-driven repository for embedded AI development.

6.4 Similarity Index Report

The similarity index of this thesis was checked using the university's designated plagiarism-detection software prior to submission. The overall similarity score obtained, along with a breakdown of matched sources, is presented below.



Figure 6.1: Similarity Index Report

15	"Explainable Intelligence in Digital Twins", Springer Science and Business Media LLC, 2020 Publication
16	Vanika Singhal, Shikha Singh, Angshul Majumdar. "How to Train Your Neural Network with Dictionary Learning", 2017 Data Compression Conference (DCC), 2017 Publication
17	Xijun Gao, Yiduo Wang, Huijie Tian. "Intelligent Monitoring and Flow Prediction for Complex Traffic Scenes Based on Deep Learning", 2026 IEEE International Conference on Power, Electronics and Green Energy (ICPEGE), 2026 Publication
18	tudr.thapar.edu:8080 Internet Source
19	Stefano Savazzi, Monica Nicoli, Vittorio Rampa. "Federated Learning with Cooperating Devices: A Consensus Approach for Massive IoT Networks", IEEE Internet of Things Journal, 2020 Publication
20	Pushpa Choudhary, Sambit Satpathy, Arvind Dagur, Dhirendra Kumar Shukla. "Recent Trends in Intelligent Computing and Communication", CRC Press, 2025 Publication
21	Smith, Sean Brendan. "Detecting Adversary Military Assets on Autonomous Edge Devices Using Deep Learning", The George Washington University, 2025 Publication

☐ Include quotes	☐ Cite	☐ Exclude metadata	☐ PDF
☐ Include bibliography	☐ Cite		

Figure 6.2: Similarity Index Report

6.5 Plagiarism Report

The full plagiarism report generated for this thesis, confirming compliance with the university's originality requirements, is attached below.

Hoor Ul Ain Amir

**BENCHMARKING COMPUTER VISION MODELS ON EDGE
COMPUTING DEVICES**

Quick Submit

Quick Submit

Information Technology University, Lahore

Document Details

Submission ID
tmcoid:1:3602893509

Submission Date
Jun 28, 2026, 6:25 PM GMT+5

Download Date
Jun 28, 2026, 6:42 PM GMT+5

File Name
Group_03_Rehan_Hafiz_FYP_Report.pdf

File Size
834.6 KB

48 Pages

11,530 Words

63,563 Characters

Figure 6.3: Plagiarism Report

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse action against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Figure 6.4: Plagiarism Report

REFERENCES

- [1] Deeplite, “Mcubench: A benchmark of tiny object detectors on mcus,” https://blog.deeplite.ai/hubfs/PDF%20Assets/Deeplite_MCUBench_ECCV2024.pdf, 2024, accessed: 2025-12-12.
- [2] D. K. Alqahtani, A. Cheema, and A. N. Toosi, “Benchmarking deep learning models for object detection on edge computing devices,” *arXiv preprint arXiv:2409.16808*, 2024, accessed: 2025-12-12. [Online]. Available: <https://www.arxiv.org/abs/2409.16808>
- [3] C. Haputhantrige, “Designing a benchmark framework for ml models in edge devices,” <https://trepo.tuni.fi/bitstream/handle/10024/157797/HaputhantrigeChalith.pdf?sequence=2>, 2024, accessed: 2025-12-12.
- [4] Y. Mou *et al.*, “μyolo: Towards single-shot object detection on microcontrollers,” *arXiv preprint arXiv:2408.15865*, 2024, accessed: 2025-12-12. [Online]. Available: <https://arxiv.org/html/2408.15865v1>
- [5] X. Wang *et al.*, “Squeezed edge yolo: Onboard object detection on edge devices,” *arXiv preprint arXiv:2312.11716*, 2023, accessed: 2025-12-12. [Online]. Available: <https://arxiv.org/html/2312.11716v1>
- [6] Tryolabs, “Machine learning on edge devices: Benchmark report,” <https://tryolabs.com/blog/machine-learning-on-edge-devices-benchmark-report>, 2024, accessed: 2025-12-12.
- [7] G. Levi and T. Hassner, “Age and gender classification using convolutional neural networks,” in *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2015, pp. 34–42, accessed: 2025-12-12. [Online]. Available: https://talhassner.github.io/home/projects/cnn_agegender/CVPR2015_CNN_AgeGenderEstimation.pdf
- [8] R. Rothe, R. Timofte, and L. V. Gool, “Deep expectation of real and apparent age from a single image without facial landmarks,” *International Journal of Computer Vision*, vol. 126, pp. 144–157, 2018, accessed: 2025-12-12. [Online]. Available: <https://link.springer.com/article/10.1007/s11263-016-0940-3>